

Dossier UML - Ecogenesia

Alexandre Perrin

Elisha Hodgson

Mathieu Le Lay

Elliot Thoribe

2 novembre 2024

Table des matières

1	Présentation et description du projet	2
	Présentation et description du projet	2
1.1	Présentation du projet	2
1.2	Description du projet	2
1.3	Diagramme de Gantt	3
1.4	Écran d'accueil	4
1.5	Design du jeu	5
1.6	Ecran de victoire	5
	Diagrammes UML	6
1.7	Diagrammes de cas d'utilisation	6
1.8	Explication des diagrammes de cas d'utilisation	7
1.9	Scénarios	8
1.10	Diagrammes de classes	8
1.11	Explications des diagrammes de classe	16
1.12	Diagramme d'états/transitions	17
1.13	Explication du diagramme d'états et transitions	18
1.14	Diagrammes d'activité	19
1.15	Explication des diagrammes d'activité	20
1.15.1	Diagramme d'interaction avec un objet	20
1.15.2	Diagramme d'utilisation d'un objet dans l'inventaire	21
1.15.3	Diagramme de déplacement du joueur	21
	Conclusion	22

Chapitre 1

Présentation et description du projet

1.1 Présentation du projet

Ecogenesia est un jeu de stratégie écologique qui s'inspire du jeu Terra Nil. Le joueur incarne un personnage humain qui revient sur Terre, longtemps après que celle-ci a été abandonnée en raison d'une pollution excessive. L'objectif principal du jeu est de restaurer les différents environnements en dépolluant les zones et en faisant revenir la faune et la flore, tout en limitant au maximum l'empreinte laissée par l'intervention du joueur. Le jeu combine des éléments de construction, de gestion des ressources et de prise de décision stratégique, offrant une expérience unique centrée sur la préservation de la nature.

1.2 Description du projet

Le jeu se déroule à travers une série de zones polluées que le joueur doit dépolluer à l'aide de divers bâtiments et infrastructures. La boucle de gameplay se compose de plusieurs étapes : arriver sur une nouvelle zone, choisir et poser les bâtiments nécessaires pour la dépollution, atteindre un niveau de dépollution de 100%, puis retirer les bâtiments pour minimiser les traces laissées. L'interface utilisateur comporte une barre de dépollution pour permettre au joueur de suivre sa progression, ainsi qu'une barre d'outils pour sélectionner et poser les bâtiments. En plus des aspects stratégiques, le jeu comporte des éléments de gestion, car le joueur doit faire des choix quant à l'emplacement des bâtiments, la quantité de ressources utilisées et le moment opportun pour les retirer.

1.3 Diagramme de Gantt

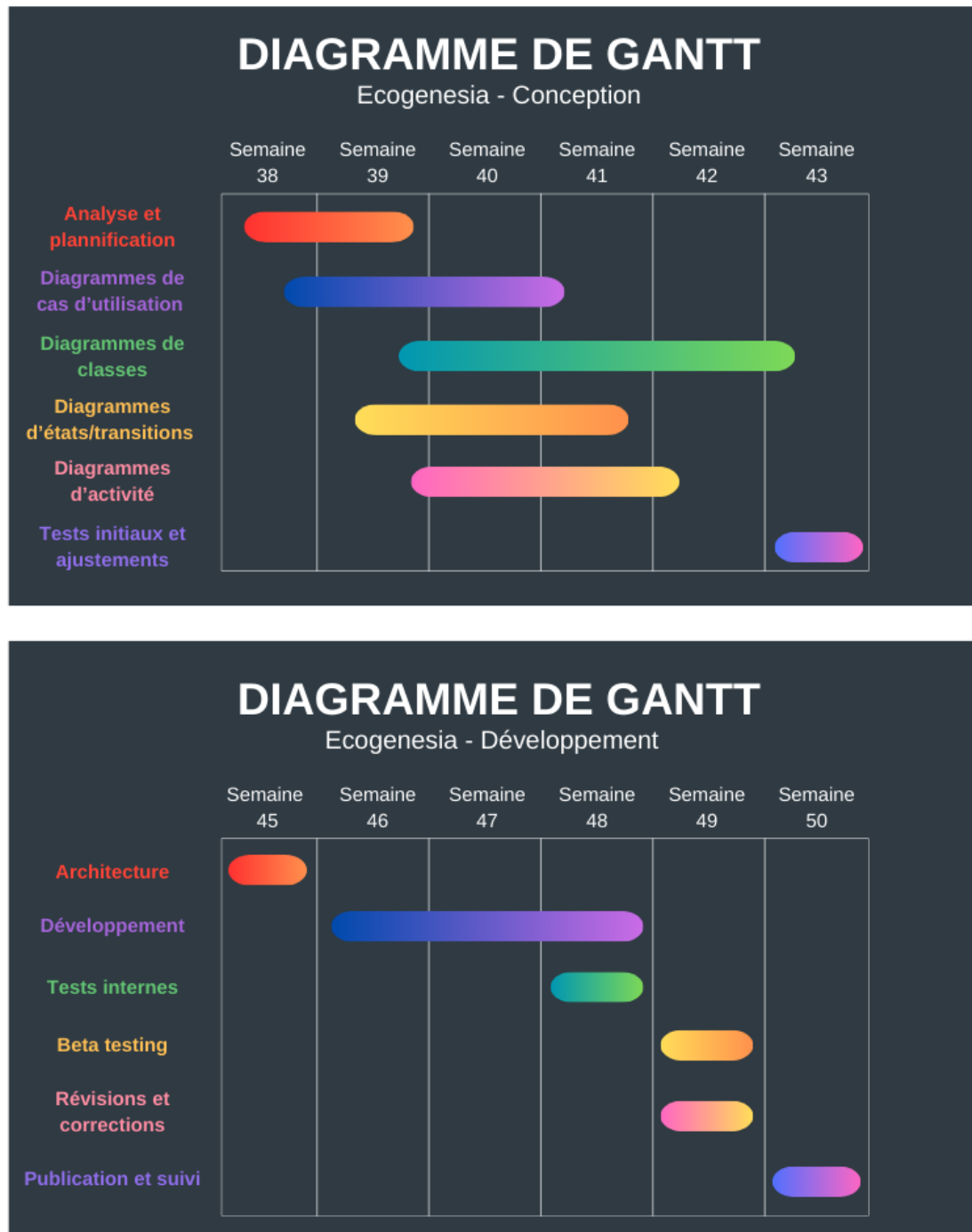


FIGURE 1.1 – Diagrammes de Gantt

1.4 Écran d'accueil

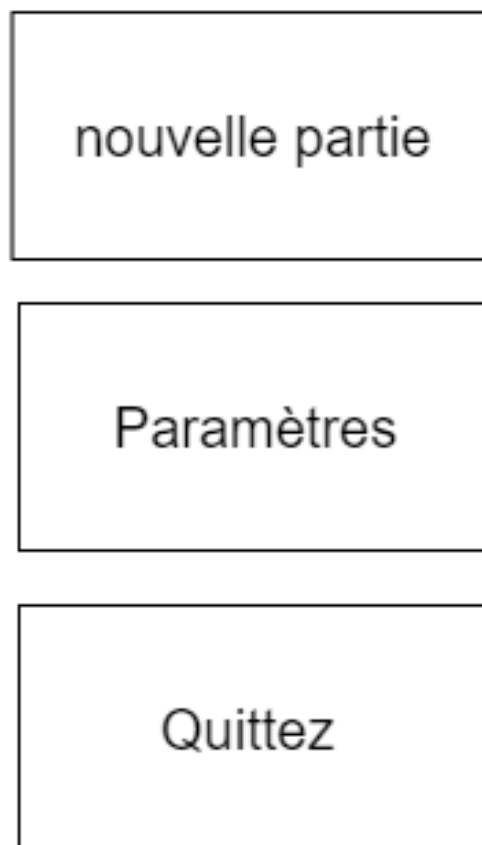


FIGURE 1.2 – Design du menu de démarrage

1.5 Design du jeu

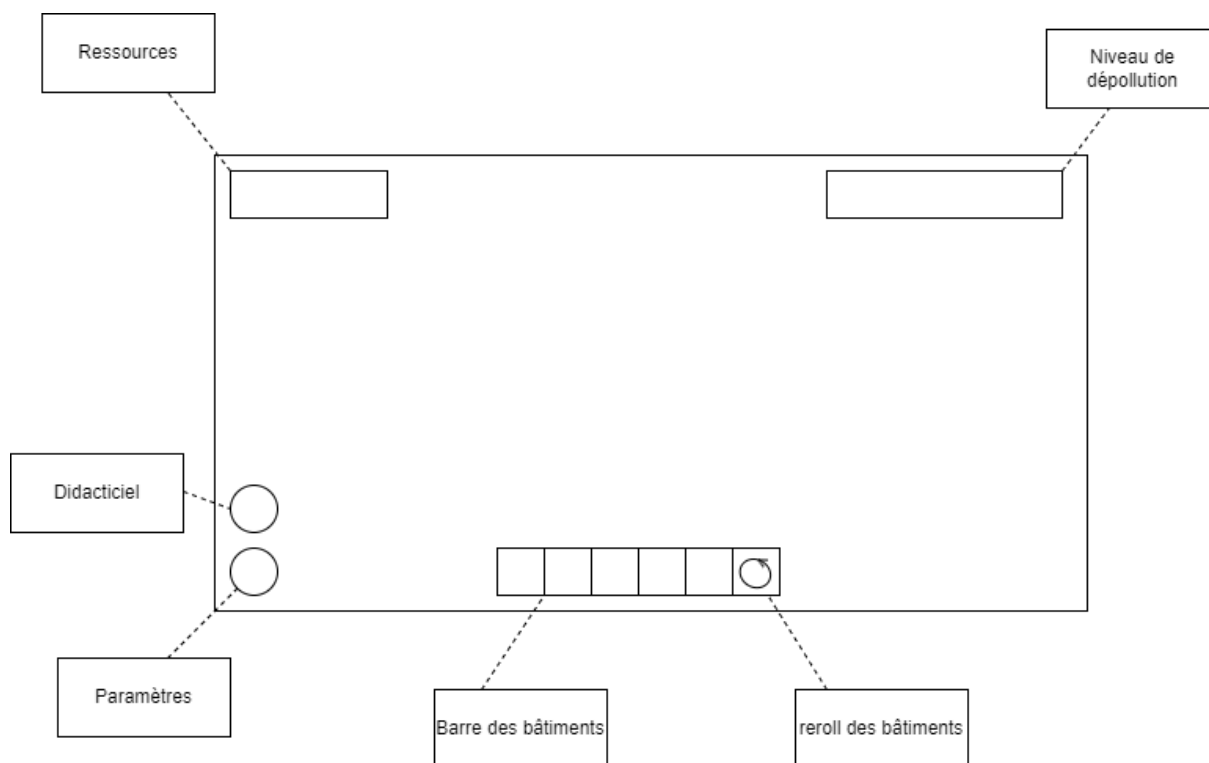


FIGURE 1.3 – Design de la fenêtre du jeu

1.6 Ecran de victoire



FIGURE 1.4 – Design de l'écran de victoire

Diagrammes UML

1.7 Diagrammes de cas d'utilisation

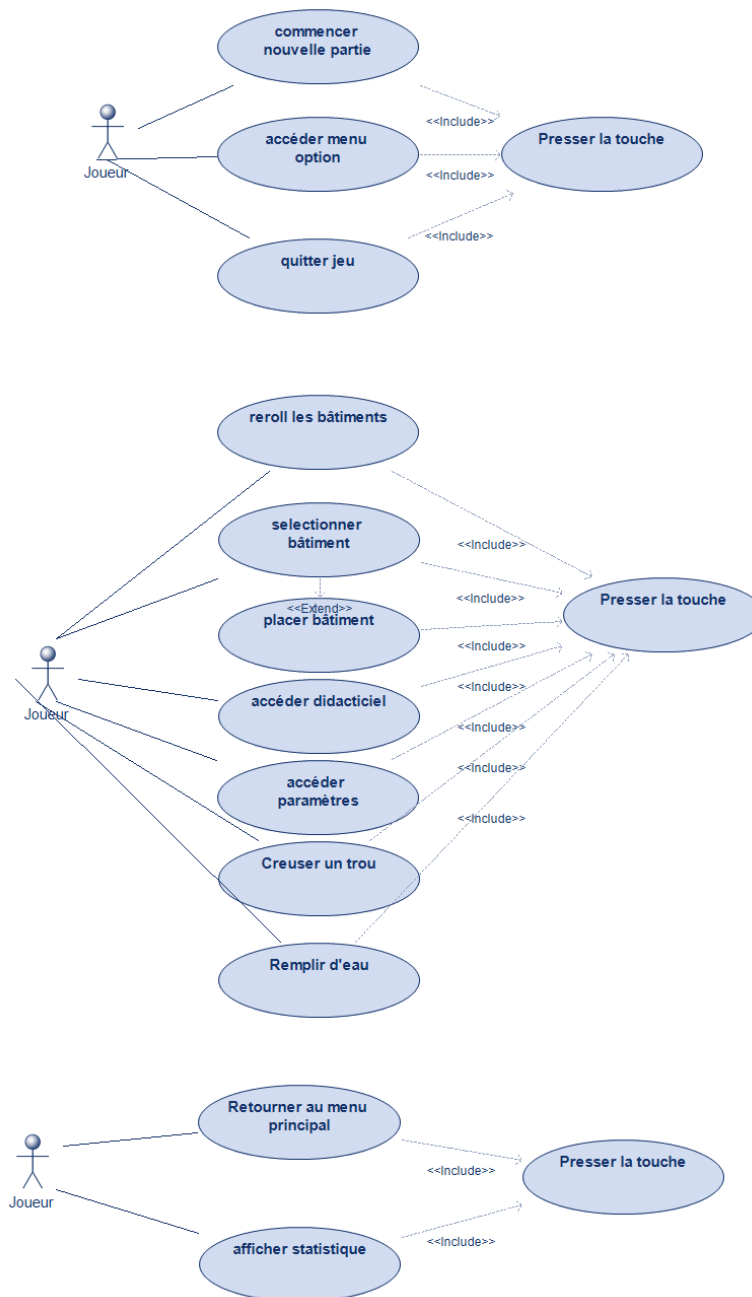


FIGURE 1.5 – Diagramme de cas d'utilisation

1.8 Explication des diagrammes de cas d'utilisation

Ce diagramme détaille les différentes actions que le joueur peut effectuer lorsqu'il interagit avec le menu et les options disponibles dans le jeu.

- **Commencer une nouvelle partie** : Le joueur arrive dans le jeu après avoir sélectionné cette option. Le jeu commence et il peut interagir avec l'environnement.
- **Accéder au menu paramètres** : Cette option permet au joueur de configurer des aspects tels que la résolution, la taille de la fenêtre du jeu, et le volume sonore.
- **Quitter le jeu** : Lorsque le joueur sélectionne cette option, le jeu se ferme et il est redirigé vers le bureau de son ordinateur.

Quand le joueur commence une nouvelle partie, il a la possibilité de réaliser plusieurs actions :

- **Reroll les bâtiments** : Le joueur peut relancer un tirage aléatoire de bâtiments si ceux fournis au début de la partie ne lui conviennent pas. Ce tirage coûte des ressources et devient plus cher à chaque tentative.
- **Sélectionner un bâtiment** : Le joueur peut choisir un bâtiment parmi ceux qui ont été tirés aléatoirement en début de partie.
- **Placer un bâtiment** : Après avoir sélectionné un bâtiment, le joueur peut le placer sur la carte de jeu à un emplacement stratégique.
- **Accéder au menu didacticiel** : Cette option permet au joueur de se familiariser avec les commandes et mécaniques du jeu.
- **Accéder au menu paramètres** : Le joueur peut également ouvrir le menu paramètres pour ajuster des éléments de configuration pendant la partie.

Une fois la partie terminée, le joueur est redirigé vers un écran de victoire, où il peut effectuer les actions suivantes :

- **Retourner au menu principal** : Cette option permet de retourner au menu principal du jeu après avoir terminé la partie.
- **Afficher les statistiques** : Le joueur peut consulter ses statistiques de jeu, comme le nombre de bâtiments placés, les ressources utilisées, etc.

1.9 Scénarios

Sommaire	Scénario nominal : Jouer au jeu
Description	1. Le joueur arrive sur le menu principal 2. Le joueur clique sur "continuer" et entre dans le jeu 3. Le joueur peut placer des bâtiments 4. Il atteint les 100% de dépollution 5. Il arrive sur un écran de fin de jeu 6. Il retourne au menu principal
Sommaire	Scénario alternatif : Une erreur impacte le jeu
Description	1. Un message d'erreur s'affiche 2. Le joueur retourne au menu principal

1.10 Diagrammes de classes

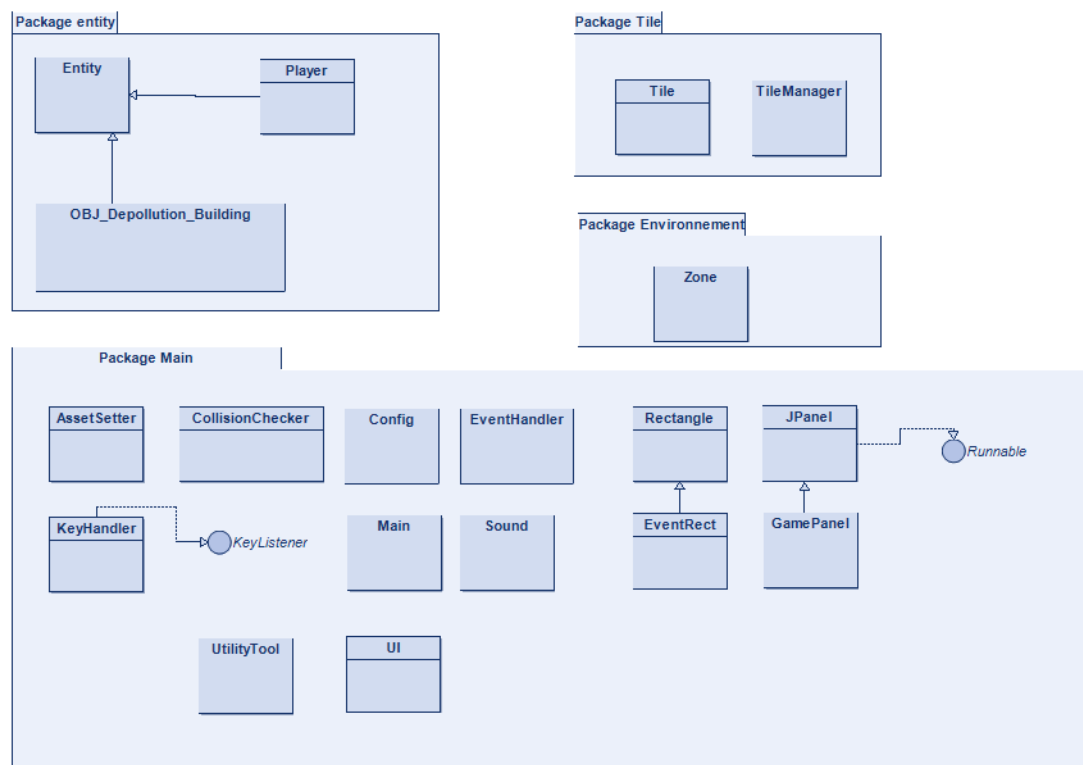


FIGURE 1.6 – Diagrammes de classes

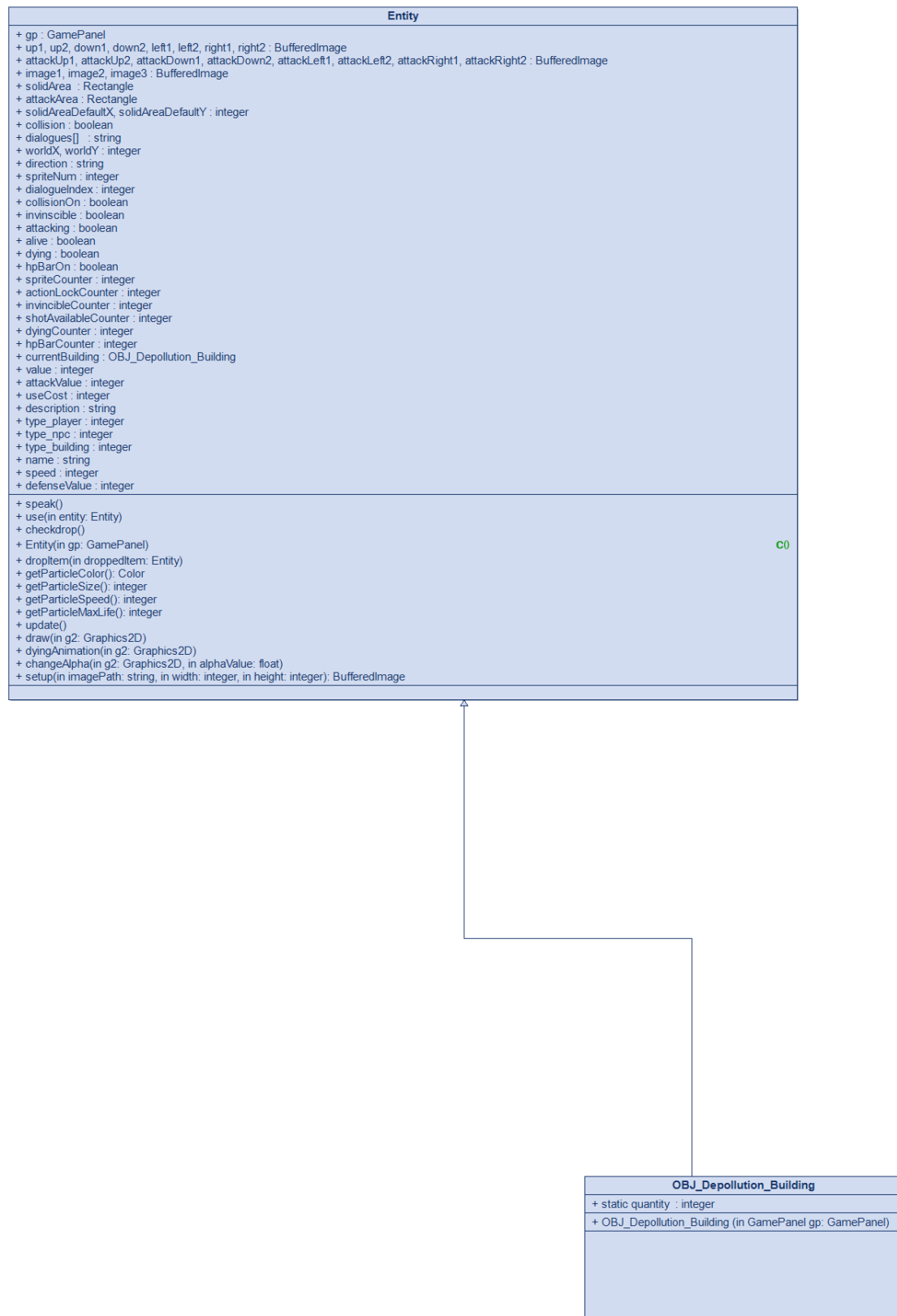


FIGURE 1.7 – Détail classe Entity

Player
keyH : KeyHandler + final screenX : integer + final screenY : integer standCounter : integer + inventory : Entity + final maxInventorySize : integer + selectedItem : Entity
+ Player(in gp: GamePanel, in keyH: KeyHandler) + setDefaultValues() + setDefaultPositions() + setItems() + getPlayerImage() + update() - handleMovement() - handleStanding() - handleInvincibility() - movePlayer() + attacking() + pickUpObject(in i: integer) - isTileWalkable(in x: integer, in y: integer): boolean + placeDepollutionBuilding() - isPositionValidAndEmpty(in x: integer, in y: integer): boolean - isPositionEmpty(in x: integer, in y: integer): boolean - placeObjectOnMap(in object: Entity, in x: integer, in y: integer) + selectItem() + draw(in g2: Graphics2D)

FIGURE 1.8 – Détail classe Player

GamePanel	
- static final serialVersionUID : long final originalTileSize : integer final scale : integer + final tileSize : integer + final maxScreenCol : integer + final maxScreenRow : integer + final screenWidth : integer + final screenHeight : integer + maxWorldCol : integer + maxWorldRow : integer + final maxMap : integer + currentMap : integer screenWidth2 : integer screenHeight2 : integer tempScreen : BufferedImage g2 : undefined + fullScreenOn : boolean FPS : integer + tileM : TileManager + KeyHandler keyH : undefined music : Sound se : Sound + CollisionChecker cChecker : CollisionChecker + aSetter : AssetSetter + ui : UI + eHandler : EventHandler config : Config gameThread : Thread + player : Player + obj[][] : Entity + npc[][] : Entity + monster[][] : Entity + iTile[][] : InteractiveTile + projectileList : Entity + particleList : Entity entityList : Entity + zones : Zone + gameState : integer + final titleState : integer + final playState : integer + final pauseState : integer + final dialogueState : integer + final characterState : integer + final optionsState : integer + final gameOverState : integer	
+ GamePanel() + void initializeZones() + void setupGame() + void retry() + void restart() + void setFullScreen() + void startGameThread() + void run() + void update() + void drawToTempScreen() + void drawToScreen() + void playMusic(in i: integer) + void stopMusic() + void playSE(in i: integer)	c0

FIGURE 1.9 – Détail classe GamePanel

Config
GamePanel gp : GamePanel
+ Config(in GamePanel gp: GamePanel) + void saveConfig() + void loadConfig()

FIGURE 1.10 – Détail classe Config

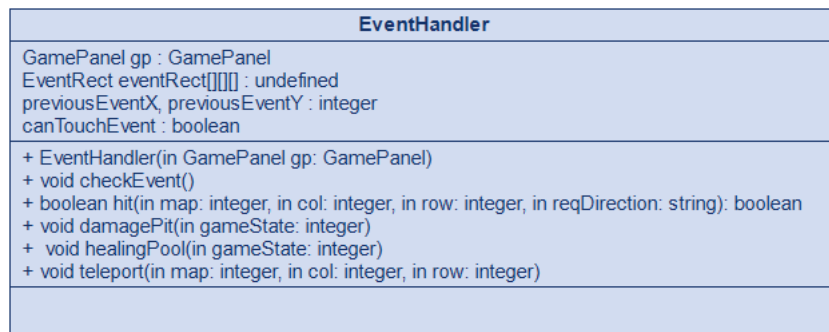


FIGURE 1.11 – Détail classe EventHandler

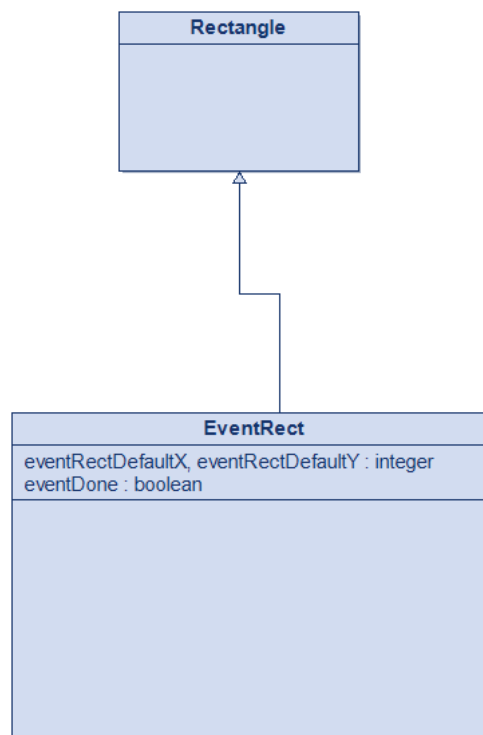


FIGURE 1.12 – Détail classe EventRect

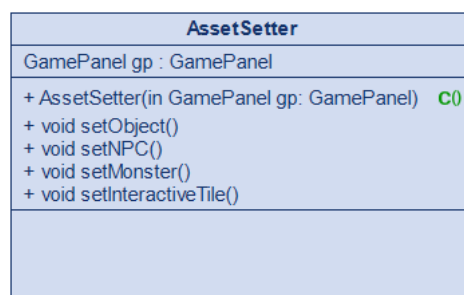


FIGURE 1.13 – Détail classe AssetSetter

CollisionChecker	
GamePanel gp : GamePanel	
+ CollisionChecker(in GamePanel gp: GamePanel) + void checkTile(in Entity entity: Entity) - boolean isValidTile(in col: integer, in row: integer): boolean + int checkObject(in Entity entity: Entity, in player: boolean): integer + int checkEntity(in Entity entity: Entity, in Entity[][] target: Entity): integer + boolean checkPlayer(in Entity entity: string): boolean	C0

FIGURE 1.14 – Détail classe CollisionChecker

KeyHandler	
GamePanel gp : GamePanel + upPressed, downPressed, leftPressed, rightPressed, enterPressed, shotKeyPressed : boolean showDebugText : boolean	
+ public KeyHandler(in GamePanel gp: GamePanel) + void keyTyped(in KeyEvent e: undefined) + void keyPressed(in KeyEvent e: undefined) + void titleState(in code: integer) + void playState(in code: integer) + void pauseState(in code: integer) + void dialogueState(in code: integer) + void optionsState(in code: integer) + void gameOverState(in code: integer) + void keyReleased(in KeyEvent e: undefined)	C0

FIGURE 1.15 – Détail classe KeyHandler

Main
static JFrame window : undefined
+ static void main(in args: string)

FIGURE 1.16 – Détail classe Main

Sound
Clip clip : undefined URL soundURL[] : undefined FloatControl fc : undefined volumeScale : integer volume : float
+ Sound() + void setFile(in i: integer) + void play() + void loop() + void stop() + void checkVolume()

FIGURE 1.17 – Détail classe Sound

Tile
+ BufferedImage image : undefined + collision : boolean

FIGURE 1.18 – Détail classe Tile

TileManager
GamePanel gp : string + tile : undefined + mapTileNum[][] : integer drawPath : boolean fileNames : undefined collisionStatus : undefined
+ TileManager(in GamePanel gp: GamePanel) C0 + void getTileImage() + void setup(in index: integer, in imageName: string, in collision: boolean) + void loadMap(in filePath: string, in map: integer) + void draw(in Graphics2D g2: undefined)

FIGURE 1.19 – Détail classe TileManager

UI	
GamePanel gp : GamePanel Graphics2D g2 : undefined Font myFont : undefined BufferedImage heart_full, heart_half, heart_blank, crystal_full, crystal_blank : undefined + messageOn : boolean ArrayList<String> message : undefined ArrayList<Integer> messageCounter : undefined + gameFinished : boolean + currentDialogue : string + commandNum : integer + titleScreenState : integer + slotCol : integer + slotRow : integer subState : integer	
+ UI(in GamePanel gp: GamePanel) + void addMessage(in text: string) + void draw(in Graphics2D g2: <no type>) + void drawPlayerLife() + void drawMessage() + void drawTitleScreen() + void drawPauseScreen() + void drawDialogueScreen() + void drawCharacterScreen() + void drawInventory() + void drawGameOverScreen() + void drawOptionsScreen() + void options_top(in frameX: integer, in frameY: integer) + void options_fullScreenNotification(in frameX: integer, in frameY: integer) + void options_control(in frameX: integer, in frameY: integer) + void options_endGameConfirmation(in frameX: integer, in frameY: integer) + int getItemIndexOnSlot(): integer + void drawSubWindow(in x: integer, in y: integer, in width: integer, in height: integer) + int getXforCenteredText(in text: string): integer + int getXforAlignToRightText(in text: string, in tailX: integer): integer	C0

FIGURE 1.20 – Détail classe UI

UtilityTool
GamePanel gp : GamePanel
+ BufferedImage scaleImage(in original: undefined, in width: integer, in height: integer): undefined

FIGURE 1.21 – Détail classe UtilityTool

Zone	
- pollutionLevel : integer - maxPollution : integer - x, y : integer - width, height : integer	
+ Zone(in x: integer, in y: integer, in width: integer, in height: integer, in maxPollution: integer) + void reducePollution(in amount: integer) + int getPollutionLevel(): integer + int getDepollutionPercentage(): integer + boolean canPlaceBuilding(in buildingX: integer, in buildingY: integer): boolean	C0

FIGURE 1.22 – Détail classe Zone

1.11 Explications des diagrammes de classe

- **Classe Entity** : La classe Entity définit l'état, le fonctionnement et les actions des objets du jeu. Elle permet notamment de gérer la taille et les interactions du joueur avec les objets. Tous les objets implémentés sont ses classes filles comme ici la classe (OBJ Depollution Building).
- **Classe Player** : La classe Player permet de définir ce que le joueur peut faire, que cela concerne ses déplacements, son inventaire ou les objets qu'il utilise. La classe Player est une classe fille de la classe Entity.
- **Classe GamePanel** : La classe GamePanel s'occupe de l'affichage des informations du jeu sur l'écran, cela comprends le Joueur et les objets. Cette classe s'occupe aussi des paramètres d'affichage (taille de la fenêtre).
- **Classe Config** : La classe Config permet de configurer différents éléments du jeu, notamment certains paramètres de la classe `GamePanel`.
- **Classe EventHandler** : La classe EventHandler gère les événements, ce qui permet d'appliquer les effets d'une action sur les différents éléments du jeu.
- **Classe EventRect** : La classe EventRect est une extension de la classe Rectangle, elle est utilisée pour gérer l'état d'un événement associé à une zone.
- **Classe AssetSetter** : La classe AssetSetter sert à initialiser différents éléments du jeu, notamment les Objets et les Interactions.
- **Classe CollisionChecker** : La classe CollisionChecker est la classe nous servant à vérifier la non superposition des éléments présents à l'écran via les collisions. Cette classe permet aussi de repositionner les différents objets/éléments du jeu si ils sont superposés.
- **Classe KeyHandler** : La classe KeyHandler est la classe qui gère le déroulé des événements et qui va indiquer au code dans quel état du jeu celui-ci se trouve (en jeu, en pause, si une victoire se passe, ...).
- **Classe Main** : La classe Main est la classe permettant de lancer le code du jeu et ses différents sous-programmes.
- **Classe Sound** : La classe Sound est la classe permettant de gérer les différents sons du jeu (son du menu, son des objets ...).
- **Classe Tile** : La classe Tile permet de créer des objets 'Tile' ayant chacun une image et un paramètre gérant leurs collisions.
- **Classe TileManager** : La classe TileManager charge les images et les données de collision des objets de la classe 'Tile', génère les cartes, et dessine efficacement les objets visibles en fonction de la position du joueur.

- **Classe UI** : La classe UI est responsable de l’affichage des éléments visuels et des messages contextuels à l’écran, tels que les dialogues, les options, les états de jeu et l’inventaire du joueur
- **Classe UtilityTool** : Cette classe permet le redimensionnement des images qui lui sont fournis dans le Jeu.
- **Classe Zone** : La classe Zone représente une zone géographique avec des propriétés de pollution et de dimensions dans l’environnement du jeu.

1.12 Diagramme d’états/transitions

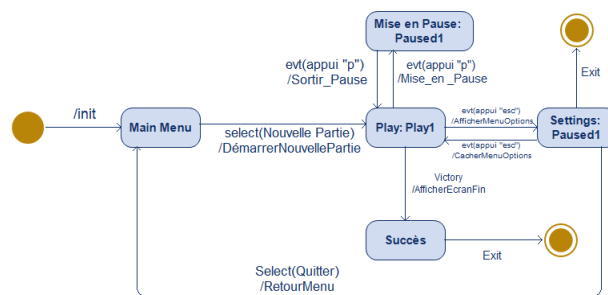


FIGURE 1.23 – Diagramme Global des états et transitions

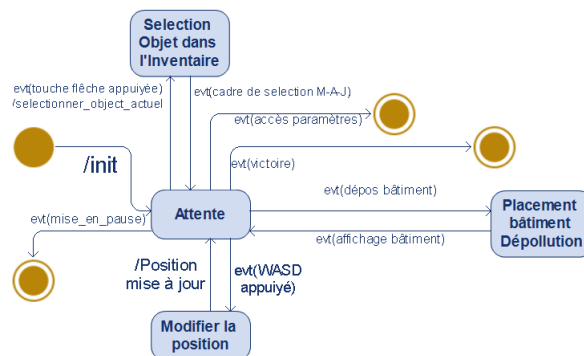


FIGURE 1.24 – Diagramme des états et transitions de l’état composite Play

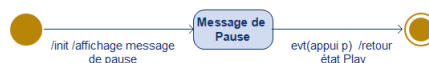


FIGURE 1.25 – Diagramme des états et transitions de l’état composite Pause

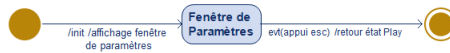


FIGURE 1.26 – Diagramme des états et transitions de l'état composite Settings

1.13 Explication du diagramme d'états et transitions

Ce diagramme d'états illustre les différentes transitions possibles entre les états principaux du jeu, à partir du menu principal, du mode de jeu et des autres étapes clés comme la pause et la réussite d'une partie.

- **Main Menu (Menu Principal)** : Le jeu commence à l'état de menu principal après l'initialisation (`/init`). À partir de cet état, le joueur peut choisir de démarrer une nouvelle partie (`select(Nouvelle Partie)`) ou de quitter le jeu (`Select(Quitter)`). Lorsqu'une nouvelle partie est sélectionnée, l'action associée est `/DémarrerNouvellePartie`, ce qui fait passer le jeu à l'état Play.
- **Play (En jeu)** : Dans cet état, le joueur est en pleine partie. Si le joueur appuie sur la touche `esc`, cela déclenche l'événement `evt(appui "esc")` qui affiche les options du menu en jeu (`/AfficherMenuOptions`). Lorsque l'utilisateur reprend le jeu, l'option `/CacherMenuOptions` est activée pour cacher le menu.
- **Game Paused (Jeu en pause)** : Lorsque le joueur appuie sur la touche `p` pendant la partie (`evt(appui "p")`), le jeu passe en état de pause (`Game Paused`). Dans cet état, le jeu reste figé jusqu'à ce que le joueur appuie de nouveau sur `p` pour sortir de la pause (`evt(appui "p") / Sortir_Pause`), retournant ainsi à l'état Play.
- **Success (Succès)** : Si le joueur remporte la partie, cela déclenche l'état `Success`, avec l'événement de victoire (`Victory`) qui affiche l'écran de fin (`/AfficherEcranFin`). Dans cet état, le joueur peut ensuite quitter le jeu en sélectionnant `Exit`, ce qui le ramène au menu principal (`RetourMenu`).
- **Settings (Paramètres)** : Depuis le menu principal ou en jeu, le joueur peut accéder aux paramètres (`Settings`) en appuyant sur une touche ou en sélectionnant cette option dans le menu. Dans cet état, le joueur peut ajuster les réglages du jeu. Le retour au jeu ou au menu principal est effectué en sélectionnant `Exit`.

Le flux commence par l'initialisation du jeu (`/init`), menant au menu principal (`Main Menu`), où le joueur peut démarrer une nouvelle partie ou accéder aux paramètres. Pendant la partie (`Play`), le joueur peut mettre le jeu en pause ou accéder aux options, et en cas de victoire, passer à l'état `Success`.

1.14 Diagrammes d'activité

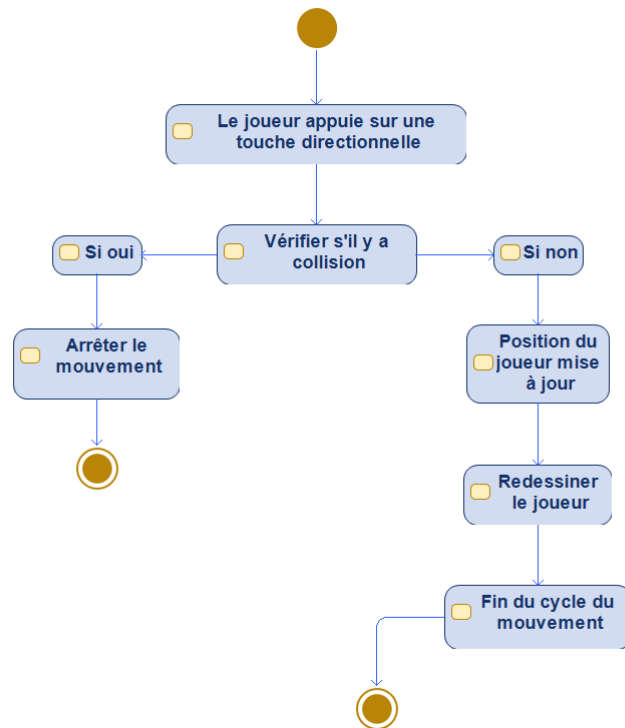


FIGURE 1.27 – Diagramme d'interaction avec un objet

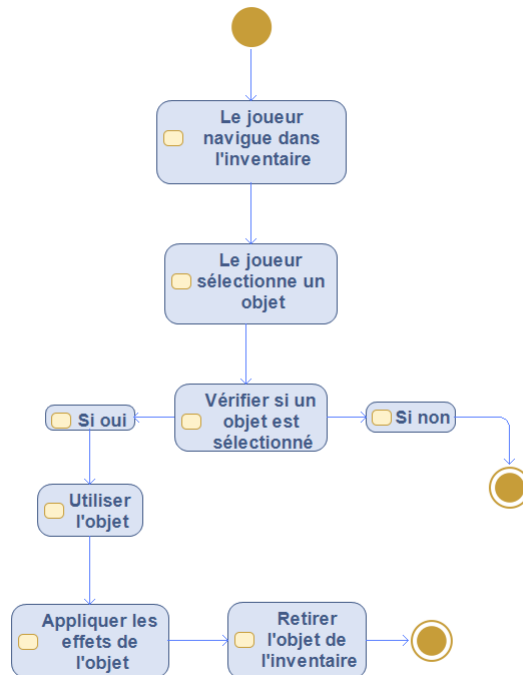


FIGURE 1.28 – Diagramme d'utilisation d'objet dans l'inventaire

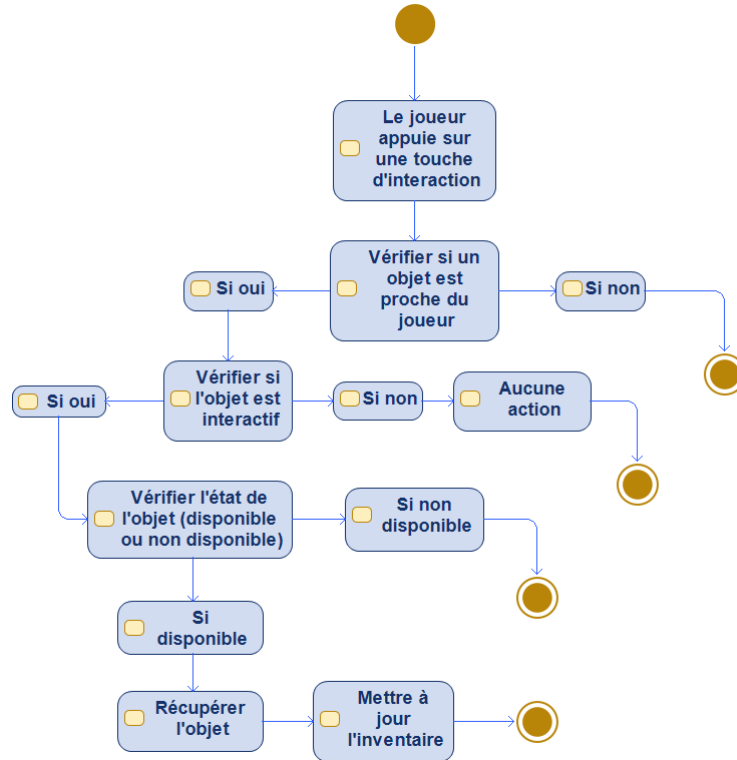


FIGURE 1.29 – Diagramme de déplacement du joueur

1.15 Explication des diagrammes d'activité

1.15.1 Diagramme d'interaction avec un objet

Ce diagramme décrit le processus lorsque le joueur interagit avec un objet dans le jeu.

- **Le joueur appuie sur une touche d'interaction** : Ce déclenchement correspond à une action du joueur pour interagir avec un objet dans son environnement.
- **Vérifier si un objet est proche du joueur** : La première vérification est de savoir si un objet interactif est à portée. Si aucun objet n'est à proximité, aucune action n'est réalisée. Sinon, on continue le processus.
- **Vérifier si l'objet est interactif** : Si un objet est proche, le système vérifie s'il est interactif. Si ce n'est pas le cas, aucune action n'est effectuée. Sinon, on passe à l'étape suivante.
- **Vérifier l'état de l'objet (disponible ou non disponible)** : Si l'objet est interactif, le système vérifie son état. S'il n'est pas disponible, le processus s'arrête ici. Si l'objet est disponible, on peut interagir avec lui.
- **Récupérer l'objet** : Si l'objet est disponible, le joueur peut le récupérer. Une fois récupéré, l'inventaire du joueur est mis à jour avec cet objet.

1.15.2 Diagramme d'utilisation d'un objet dans l'inventaire

Ce diagramme détaille le processus lorsque le joueur souhaite utiliser un objet de son inventaire.

- **Le joueur navigue dans l'inventaire** : Le joueur commence par naviguer dans son inventaire pour trouver l'objet qu'il souhaite utiliser.
- **Le joueur sélectionne un objet** : Une fois que l'objet est trouvé, il le sélectionne.
- **Vérifier si un objet est sélectionné** : Si aucun objet n'est sélectionné, l'action s'arrête ici. Sinon, l'objet peut être utilisé.
- **Utiliser l'objet** : Si l'objet est sélectionné, le joueur peut l'utiliser. Après l'utilisation, l'effet de l'objet est appliqué (par exemple, un objet de soin pourrait restaurer de la santé).
- **Appliquer les effets de l'objet** : L'effet de l'objet est alors appliqué au joueur. Une fois utilisé, l'objet est retiré de l'inventaire.

1.15.3 Diagramme de déplacement du joueur

Ce diagramme montre les étapes lorsqu'un joueur se déplace dans le jeu.

- **Le joueur appuie sur une touche directionnelle** : Le joueur initie un déplacement en appuyant sur une touche directionnelle (par exemple, les flèches de direction).
- **Vérifier s'il y a collision** : Lorsque le joueur se déplace, le système vérifie s'il y a une collision avec un objet ou un mur. Si oui, le mouvement est arrêté.
- **Arrêter le mouvement (en cas de collision)** : Si une collision est détectée, le joueur ne peut pas continuer dans cette direction, et le mouvement est interrompu.
- **Position du joueur mise à jour** : Si aucune collision n'est détectée, la position du joueur est mise à jour en fonction de la direction choisie.
- **Redessiner le joueur** : Après la mise à jour de la position, le joueur est redessiné à son nouvel emplacement dans l'environnement de jeu.
- **Fin du cycle de mouvement** : Une fois que le joueur a été redessiné, le cycle de mouvement est terminé, et le joueur peut initier une nouvelle action ou un nouveau déplacement.

Conclusion

Ecogenesia est un jeu immersif conçu pour pousser les joueurs à réfléchir aux conséquences de leurs actions sur l'environnement. Il place les joueurs face à des choix où chaque décision a un impact sur l'écosystème. Les joueurs doivent gérer des ressources et anticiper les effets de leurs actions, mettant en avant l'importance de maintenir un équilibre durable. Le jeu se distingue par des mécanismes diversifiés qui simulent des efforts de dépollution et de préservation écologique, encourageant des approches réfléchies et responsables. Ecogenesia invite à se plonger dans une expérience qui sensibilise aux enjeux environnementaux tout en offrant un défi engageant.